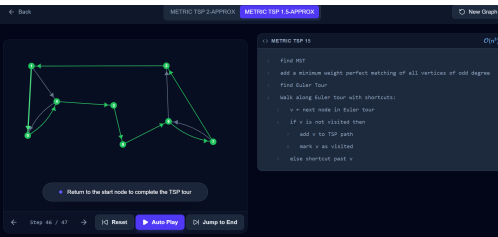


15 Approximation für metrisches TSP

Wie viele ZHKs wenn wir einen Knoten/Kante wegnehmen?

Korrektur Laufzeit $O(|V| |E|)$

• Auf Notizen von vorheriger Woche



Hopcroft-Karp Algorithmus

- Findet ein kardinalitätsmaximales Matching für einen bipartiten Graph
(viele Matching-Probleme sind bipartit)

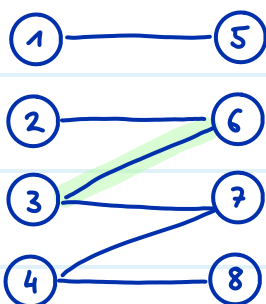
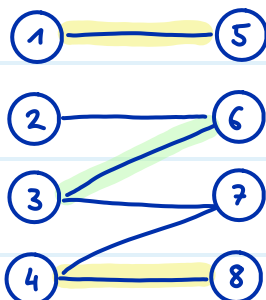
⊕

MAXIMAL_MATCHING ($G = (A \oplus B, E)$) (Hopcroft und Karp)

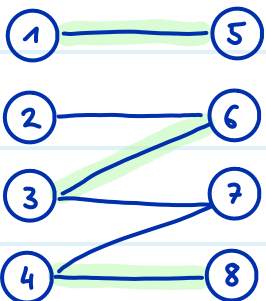
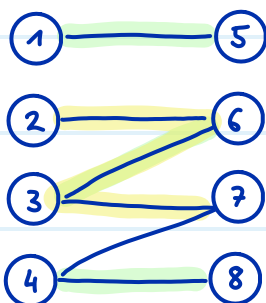
- 1: $M := \{e\}$ für irgendeine Kante $e \in E$.
 - 2: **while** es gibt noch augmentierende Pfade **do**
 - 3: $k :=$ Länge eines kürzesten augmentierenden Pfades
 - 4: Finde eine inklusionsmaximale Menge S von paarweise disjunkten augmentierenden Pfaden der Länge k .
 - 5: **for all** P aus S **do**
 - 6: $M := M \oplus P$. // *augmentiere entlang der Pfade aus S*
 - 7: **return** M
-

- "inklusionsmaximal" = wir können keinen weiteren Pfad zu S hinzufügen
- "paarweise disjunkt" = es gibt keine zwei Pfade, die beide denselben Knoten v enthalten
- "augmentierend" = Endpunkte des Pfades sind nicht überdeckt, und die Kanten wechseln ab zwischen Matchingkanten und nicht Matchingkanten
- "Länge" = Anzahl Kanten ($\neq$ Anzahl Knoten)

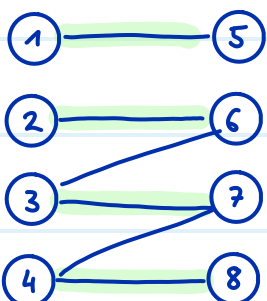
Bsp:

1: $M := \{e\}$ für irgendeine Kante $e \in E$.2: **while** es gibt noch augmentierende Pfade ✓ **do**3: $k :=$ Länge eines kürzesten augmentierenden Pfades $= 1$ 4: Finde eine inklusionsmaximale Menge S von paarweise disjunkten augmentierenden Pfaden der Länge k .

$$S = \{(1,5), (4,8)\}$$

es gäbe auch andere Möglichkeiten, z.B. $S = \{(1,5), (4,7)\}$ 5: **for all** P aus S **do** $P_1 = (1,5), P_2 = (4,8)$ 6: $M := M \oplus P$. // augmentiere entlang der Pfade aus S 2: **while** es gibt noch augmentierende Pfade ✓ **do**3: $k :=$ Länge eines kürzesten augmentierenden Pfades $= 3$ 4: Finde eine inklusionsmaximale Menge S von paarweise disjunkten augmentierenden Pfaden der Länge k .

$$S = \{(2,6), (3,7)\}$$

5: **for all** P aus S **do** $P = (2,6,3,7)$ 6: $M := M \oplus P$. // augmentiere entlang der Pfade aus S 2: **while** es gibt noch augmentierende Pfade ✗ falsch **do**7: **return** M Satz von Berge garantiert, dass M kardinalitätsmaximal ist

Feststellungen:

1) Augmentierende Pfade haben immer ungerade Längen

→ weil die erste und letzte Kante nicht aus M sind und die Kanten dazwischen aus M und nicht M abwechseln

2) Die Knoten von einem augmentierenden Pfad wechseln zwischen A und B ab

→ weil G bipartit ist, d.h. jede Kante hat genau einen Endpunkt in A und einen in B

3) Genau ein Endknoten von einem augmentierenden Pfad ist in A und der andere in B

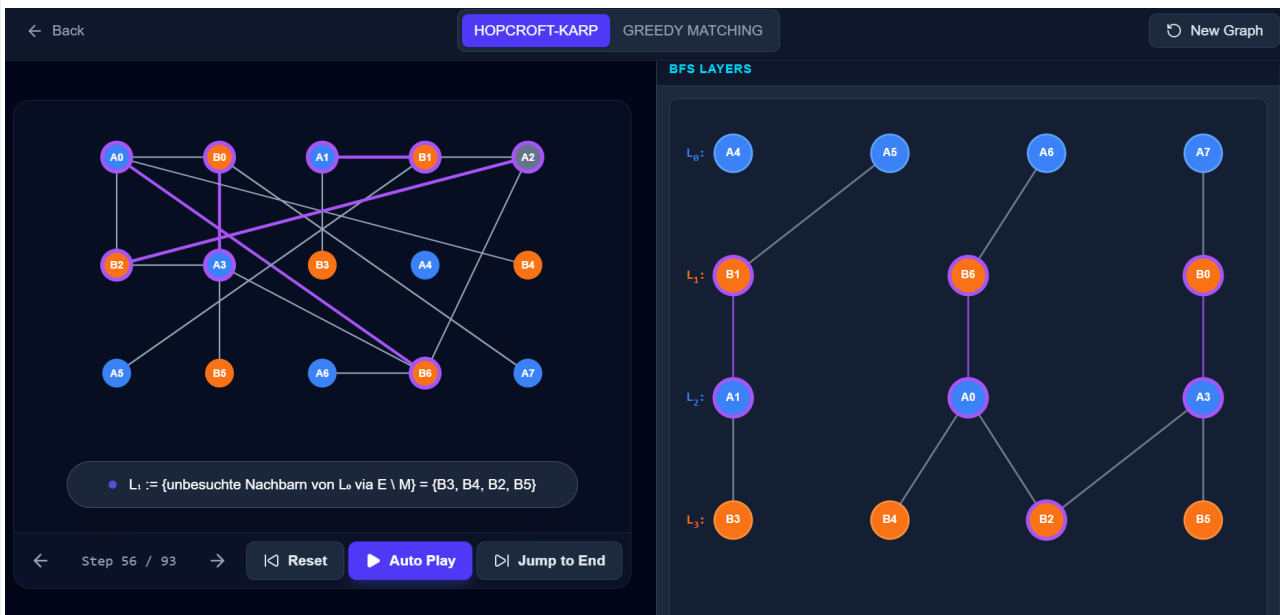
→ weil die Länge ungerade ist (1) und Knoten aus A und B abwechseln (2)

4.) $L_0 = \{v \in A \mid v \text{ ist nicht überdeckt}\}$ enthält alle Startknoten aller möglichen augmentierenden Pfade → folgt aus (3)

$L_1 := \{v \in B \mid v \text{ ist über eine Kante aus nicht } M \text{ mit einem Knoten aus } L_0 \text{ verbunden}\}$

L_2

:



Laufzeit: $O(\sqrt{|V|} |E|)$

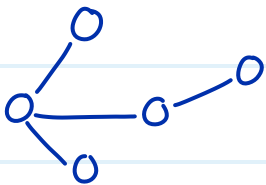
- Beweis dazu sollte man verstehen können, wird aber hier aus Zeitgründen ausgelassen
- Wenn ihr diesen Algorithmus erklären könnt, seid ihr gut vorbereitet für Matchings!
- Wichtigster Algorithmus bis jetzt

Färbungen

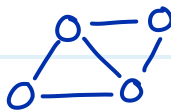
$[k] = \{1, 2, \dots, k\}$ sind die Farben

- Eine **Färbung** eines Graphen mit k Farben ist eine Abbildung $c: V \rightarrow [k]$, sodass keine zwei Nachbarknoten dieselbe Farbe haben, d.h. $\forall \{u, v\} \in E: c(u) \neq c(v)$
- Die **chromatische Zahl** $\chi(G)$ ist die minimale Anzahl an Farben, die für eine Färbung von G benötigt werden

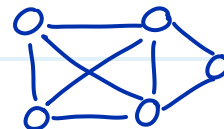
Bsp



$\chi(G) =$ _____

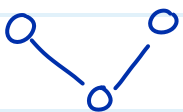


$\chi(G) =$ _____

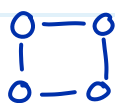


$\chi(G) =$ _____

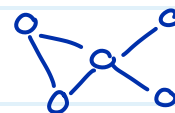
G ist **bipartit** falls man V in zwei Mengen A und B aufteilen kann, sodass alle Kanten genau einen Endpunkt in A und einen in B haben



bipartit? $\rightarrow$ _____



bipartit? $\rightarrow$ _____

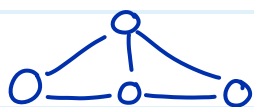


bipartit? $\rightarrow$ _____

G ist bipartit $\Leftrightarrow G$ ist 2 färbbar

auch independent sets genannt

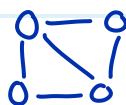
G ist k -partit falls man V in k Mengen $V_1, V_2, \dots, V_k$ aufteilen kann, sodass für alle Kanten die beiden Endpunkte nicht in der selben Menge V_i liegen.



V_1
 V_2
 V_3

2-partit? $\rightarrow$

3-partit? $\rightarrow$



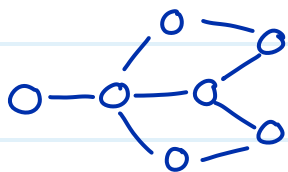
V_1
 V_2
 V_3

2-partit? $\rightarrow$

3-partit? $\rightarrow$

G ist k -partit $\Leftrightarrow G$ ist k färbbar

G ist bipartit $\Leftrightarrow$ es gibt keinen Kreis ungerader Länge (Beweis ausgelassen)

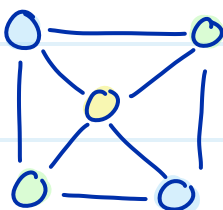


$\rightarrow$ ist bipartit $\rightarrow$ ist 2 färbbar

Eine Farbkasse ist eine Menge von Knoten derselben Farbe (= eine Menge V_i der Partition)

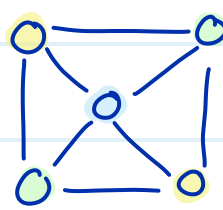
Wir dürfen Farbklassen tauschen.

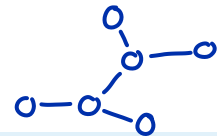
V_1 V_2 V_3



V_1 und V_2 tauschen
 $\longrightarrow$

V_1 V_2 V_3



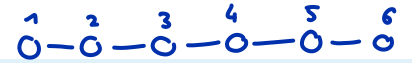


Was ist $\chi(T)$ für einen Baum T ?

→

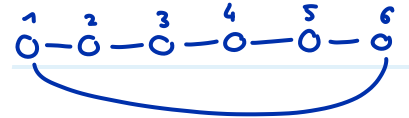
Was ist $\chi(P)$ für einen Pfad P ?

→



Was ist $\chi(C_n)$ für einen Kreis C_n gerader Länge?

→



Was ist $\chi(C_n)$ für einen Kreis C_n ungerader Länge?

→

Was ist $\chi(K_n)$ für den vollständigen Graphen mit n Knoten?

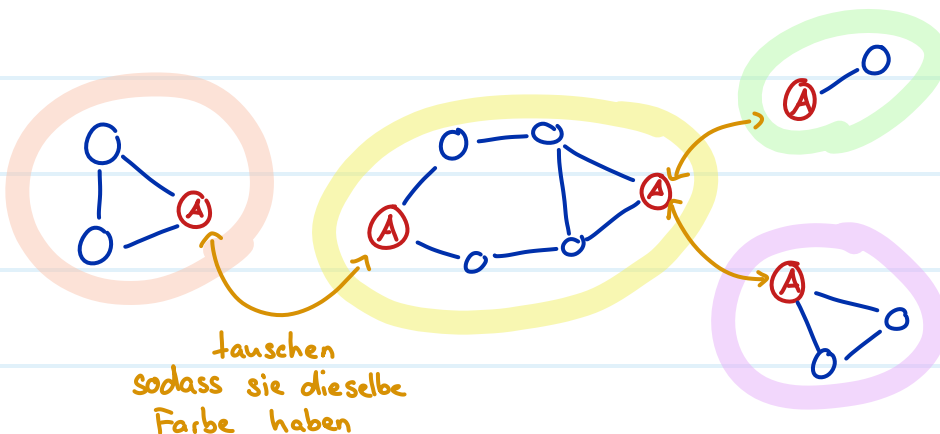
→

Was ist $\chi(G)$ falls jeder einzelne Block von G k -färbbar ist?

→



Blöcke überschneiden sich nur in Artikulationsknoten



kann keine Konflikte
geben weil der Block-
Graph immer ein
Baum ist

Ein Chemiker muss fünf verschiedene Chemikalien (A, B, C, D, E) in Sicherheitsbehältern lagern. Manche Chemikalien reagieren heftig miteinander und dürfen daher **nicht im selben Behälter** aufbewahrt werden:

- A verträgt sich nicht mit B und C.
- B verträgt sich nicht mit A, C und D.
- C verträgt sich nicht mit A, B und E.
- D verträgt sich nicht mit B.
- E verträgt sich nicht mit C.

Matching vs Färbung

Was ist die kleinste Anzahl an Sicherheitsbehältern, die der Chemiker kaufen muss?

Das ETH Prüfungsteam muss die Prüfungstermine festlegen. Ein Schüler darf jeweils nur eine Prüfung pro Tag schreiben.

- Anna hat die Fächer PProg, DDCA, Analysis
- Peter hat die Fächer DDCA, EProg, A&W
- Max hat die Fächer EProg, LinAlg
- Theo hat die Fächer A&W, LinAlg
- Luna hat die Fächer DDCA, PProg

Wie viele Prüfungstage muss es mindestens geben?



Sei G 2-färbbar. Wie können wir eine Färbung berechnen?

→

"Gibt es eine Färbung mit $\leq k$ Farben?" ist NP-vollständig für $k \geq 3$

Greedy-Färbung

GREEDY-FÄRBUNG (G)

- 1: wähle eine beliebige Reihenfolge der Knoten: $V = \{v_1, \dots, v_n\}$
- 2: $c[v_1] \leftarrow 1$
- 3: for $i = 2$ to n do
- 4: $c[v_i] \leftarrow \min\{k \in \mathbb{N} \mid k \neq c(u) \text{ für alle } u \in N(v_i) \cap \{v_1, \dots, v_{i-1}\}\}$

= kleinste Farbe, die kein Nachbar von v_i hat

- braucht höchstens $\Delta(G) + 1$ Farben (weil wenn wir v_i färben, gibt es höchstens $\Delta(G)$ gefärbte Nachbarn)
maximaler Grad
- Für jeden Graphen gilt deshalb: $\chi(G) \leq \Delta(G) + 1$
- Laufzeit $O(n+m)$ Achtung Skript: $O(m)$ falls zusammenhängend

• Wie gut der Greedy Algorithmus ist hängt sehr von der Reihenfolge ab

Verbesserung: Heuristik zur Bestimmung der Knotenreihenfolge
= Abschätzung

Idee: Schwierige Knoten zuerst färben, einfachere erst am Schluss
viele gefärbte Nachbarn wenig gefärbte Nachbarn

The screenshot shows a graph with 15 nodes (A-O) and a simulation of the Smallest Last Coloring algorithm. The interface includes a 'Back' button, tabs for 'SMALLEST LAST COLORING' and 'GREEDY COLORING', and a 'New Graph' button. The graph is displayed on the left, with nodes A through O. Below the graph, there are color selection buttons (1-6) and a 'Beginne Schleife: i = 15 bis 1' button. The right panel shows the algorithm's pseudocode and a legend for the graph's state (Current, Colored, Pending).

SMALLEST LAST COLORING $O(n+m)$

```

1  for i = n down to 1 do
2     $v_i \leftarrow$  vertex with minimum degree in current G
3    Remove  $v_i$  and incident edges from G
4     $c[v_i] \leftarrow 1$ 
5    for i = 2 to n do
6       $c[v_i] \leftarrow \min\{k \in \mathbb{N} \mid k \neq c(u) \forall u \in N(v_i) \cap \{v_1, \dots, v_{i-1}\}\}$ 

```

SMALLEST-LAST ORDER

Determining Order...

 Current
 Colored
 Pending

Step 0 / 46 Reset Auto Play Jump to End

Sei $k \in \mathbb{N}$. Falls jeder induzierte Subgraph einen Knoten mit Grad höchstens k hat, dann gibt es eine Färbung mit $k+1$ Farben.

Laufzeit $O(|V|+|E|)$, Beweis unwichtig

Achtung Skript: $O(m)$ falls zusammenhängend